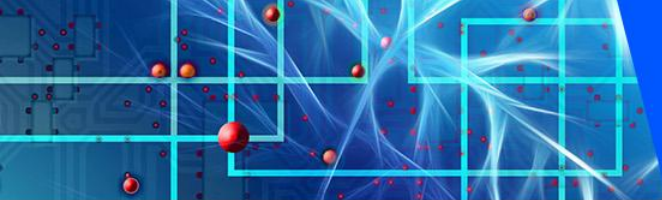


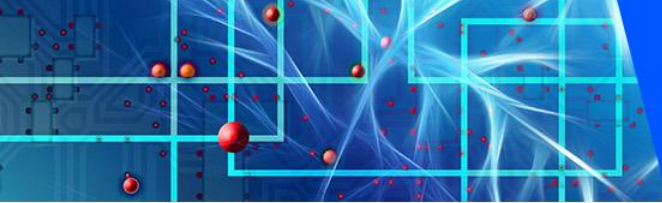
Modelos de Aprendizaje Automático

- Modelos geométricos: Pueden tener una, dos o múltiples dimensiones. Si hay un borde de decisión lineal entre las clases, se dice que los datos son linealmente separables.
- Modelos probabilísticos: Intentan determinar la distribución de probabilidades descriptora de la función que enlaza a los valores de las características con valores determinados. Uno de los conceptos claves para desarrollar modelos probabilísticos es la estadística bayesiana.
- Modelos lógicos: Transforman y expresan las probabilidades en reglas organizadas en forma de árboles de decisión.



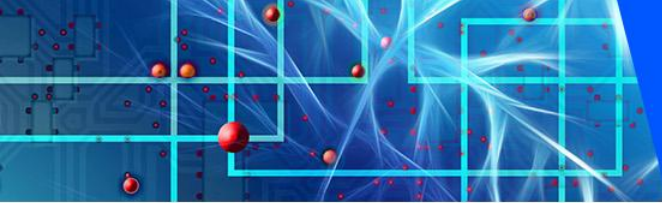
Tipos de algoritmos (I)

1. Aprendizaje supervisado: El algoritmo produce una función que establece una correspondencia entre las entradas y las salidas deseadas del sistema. Por ejemplo, los problemas de clasificación.
2. Aprendizaje no supervisado: No se tiene información sobre las categorías de los ejemplos. El sistema tiene que ser capaz de reconocer patrones a partir de datos nuevos. Un ejemplo son los mapas auto-organizados, donde elementos de entrada similares tienden a excitar neuronas adyacentes.
3. Aprendizaje semi-supervisado: Son entrenados inicialmente, aunque después se pueden readaptar.



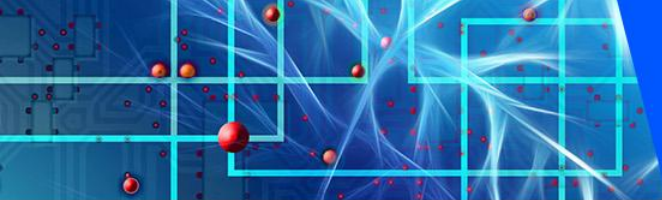
Tipos de algoritmos (II)

4. Aprendizaje por refuerzo: Un agente inteligente debe aprender cómo se comporta el entorno mediante recompensas (refuerzos) o castigos (retroalimentación). Los tres métodos principales son: la Programación Dinámica, los métodos de Monte Carlo y el aprendizaje de Diferencias Temporales.
5. Transducción: Trata de predecir las categorías de los futuros ejemplos basándose en los ejemplos de entrada, sus respectivas categorías y los ejemplos nuevos al sistema.
6. Aprendizaje multi-tarea: Usan conocimiento previamente aprendido por el sistema de cara a enfrentarse a problemas parecidos a los ya vistos.



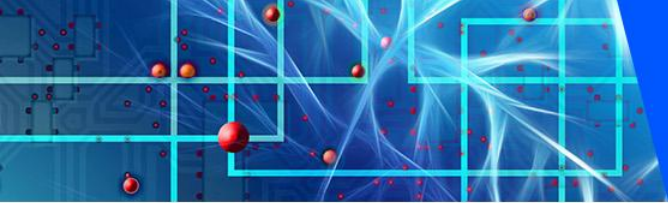
Técnicas de clasificación (I)

1. Árboles de decisiones
2. Reglas de asociación: Procuran descubrir relaciones interesantes entre variables.
3. Algoritmos genéticos: Procesos de búsqueda heurística que simulan la selección natural.
4. Redes neuronales artificiales: Las redes de neuronas artificiales (RNA) son un paradigma de aprendizaje automático inspirado en las neuronas de los sistemas nerviosos de los animales. Se trata de un sistema de enlaces de neuronas que colaboran entre sí para producir un estímulo de salida.



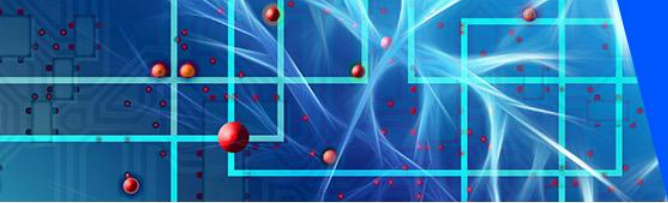
Técnicas de clasificación (II)

5. Máquinas de vectores de soporte: Los algoritmos de MVS usan un conjunto de ejemplos de entrenamiento clasificado en dos categorías para construir un modelo que prediga si un nuevo ejemplo pertenece a una u otra de dichas categorías.
6. Algoritmos de agrupamiento: El agrupamiento es un método de aprendizaje no supervisado y es una técnica muy popular de análisis estadístico de datos. Las técnicas de agrupamiento hacen inferencias diferentes sobre la estructura de los datos; se guían usualmente por una medida de similitud específica y por un nivel de compactamiento interno (similitud entre los miembros de un grupo) y la separación entre los diferentes grupos.
7. Redes bayesianas: Modelo probabilístico que representa una serie de variables de azar y sus independencias condicionales a través de un grafo acíclico dirigido.



Regresión (I)

- Regresión Lineal (I)
 - La regresión lineal es un algoritmo de aprendizaje supervisado que se utiliza en Machine Learning y en estadística.
 - En estadística, la regresión lineal es un enfoque para modelar la relación entre una variable escalar dependiente " y " y una o más variables explicativas denotadas por " X ".



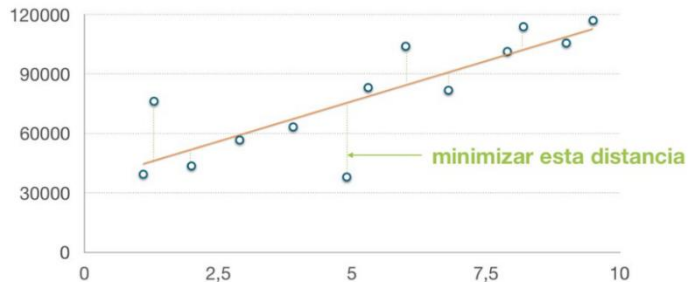
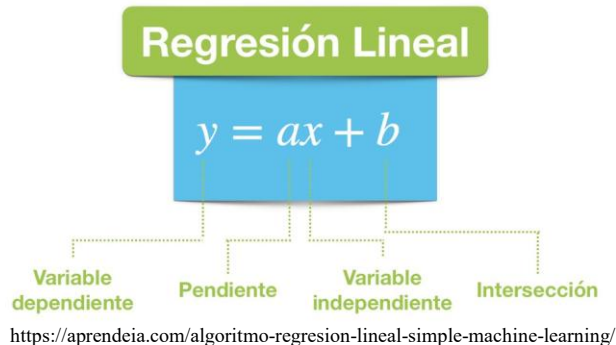
Regresión (II)

- Regresión Lineal (II)
 - La regresión lineal es una técnica paramétrica utilizada para predecir variables dependientes continuas a partir de un conjunto de variables independientes. Es paramétrica por naturaleza porque hace ciertas suposiciones basadas en el conjunto de datos. Si el conjunto de datos se ajusta a esas suposiciones, la regresión arroja buenos resultados. En caso contrario, tiene dificultades para ofrecer buenos resultados.

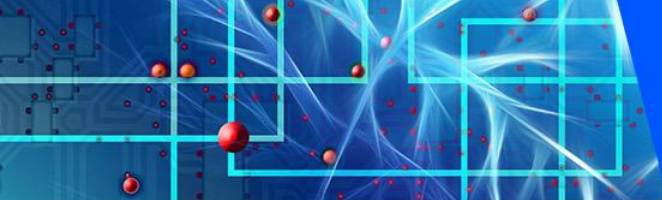
Regresión (III)

- Regresión Lineal (III):

- y – es la variable dependiente o la variable a predecir.
- x – es la variable independiente o la variable que usamos para hacer una predicción.
- a – es la pendiente o el valor que debe ser determinado, se le conoce como coeficiente.
- b – es la constante que debe ser determinada, se le conoce como intercepto porque cuando x es igual a 0, entonces $y = b$.
- El objetivo con Regresión Lineal Simple es minimizar la distancia vertical entre todos los datos y nuestra línea, por lo tanto, para determinar la mejor línea, debemos minimizar la distancia entre todos los puntos y la distancia de nuestra línea. Es utilizando el criterio de mínimos cuadrados para reducir el error.



<https://aprendeia.com/algorithmo-regresion-lineal-simple-machine-learning/>



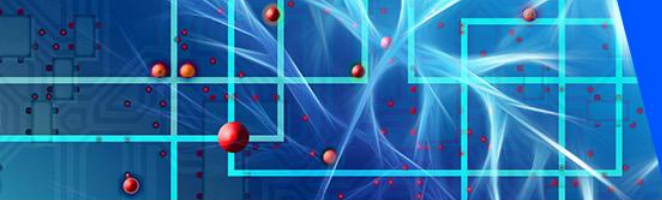
Regresión (IV)

- Regresión Lineal (IV):

- La técnica de mínimos cuadrado intenta reducir la suma de los errores al cuadrado, buscando el mejor valor posible de los coeficientes de regresión.
- Los mínimos cuadrados no es la única técnica para usar en Regresión Lineal pero se selecciona debido:
- Utiliza un error cuadrado que tiene buenas propiedades matemáticas, por lo que es más fácil diferenciar y calcular el descenso del gradiente.
- Es fácil de analizar y computacionalmente más rápido, es decir, puede aplicarse rápidamente a conjuntos de datos que tienen miles de características.
- La interpretación es mucho más fácil que otras técnicas de regresión.

$$a = \frac{\sum (x_i - x_{media})(y_i - y_{media})}{\sum (x_i - x_{media})^2}$$

$$b = y_{media} - a(x_{media})$$



Regresión (V)

- Regresión Lineal (V):

- La regresión es una técnica paramétrica, por lo que hace suposiciones, veamos algunas de ellas:
 - Existe una relación lineal, aditiva, entre las variables dependiente e independiente. Por lineal, se refiere a que un cambio en la variable dependiente para un cambio unitario en la variable independiente es constante. Por aditivo, se refiere a que el efecto de "x" e "y" es independiente de otras variables.
 - No debe haber correlación entre las variables independientes. La presencia de correlación en las variables independientes conduce a la multicolinealidad. Si las variables están correlacionadas, resulta muy difícil para el modelo determinar la verdad de las variables independientes sobre las variables dependientes.
 - Los términos de error deben tener una varianza constante y no deben estar correlacionados. La presencia de correlación en los términos de error se conoce como autocorrelación y afecta drásticamente a los coeficientes de regresión y a los valores del error estándar, ya que se basan en el supuesto de términos de error no correlacionados.
 - La variable dependiente y los términos de error deben tener una distribución normal.

Regresión (VI)

- Regresión Lineal (VI):
 - Puedes realizar la práctica en Google Colab: <https://colab.research.google.com/>
 - Regresión lineal sobre el precio de la vivienda en Boston a partir del número de habitaciones de una vivienda
 - Lo primero: Cargar el modelo y seleccionar X e Y

```
#Se importan la librerías a utilizar
import numpy as np
from sklearn import datasets, linear_model
from sklearn.datasets import fetch_openml
import matplotlib.pyplot as plt

# Cargar el dataset Boston Housing desde OpenML
boston = fetch_openml(name='boston', version=1, as_frame=False)

#Verifico la información contenida en el dataset
print('Información en el dataset:')
print(boston.keys())
#Verifico las características del dataset
print('Características del dataset:')
print(boston.DESCR)
#Verifico la cantidad de datos que hay en los dataset
print('Cantidad de datos:')
print(boston.data.shape)
```

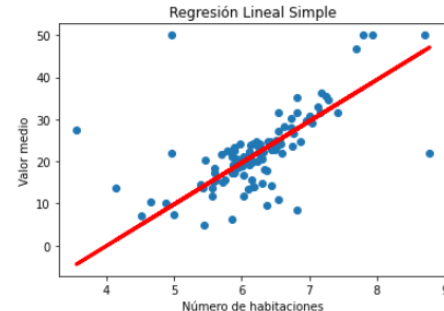
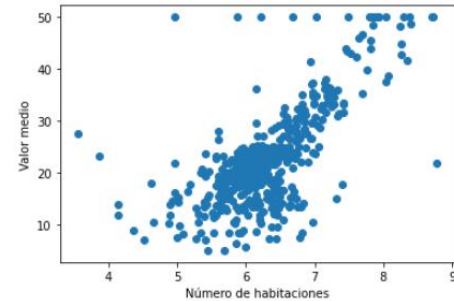
```
#Verifico la información de las columnas
print('Nombres columnas:')
print(boston.feature_names)
#Seleccionamos solamente la columna 5 del dataset
X = boston.data[:, np.newaxis, 5]
#Defino los datos correspondientes a las etiquetas
y = boston.target
#Graficamos los datos correspondientes
plt.scatter(X, y)
plt.xlabel('Número de habitaciones')
plt.ylabel('Valor medio')
plt.show()
```

Regresión (VII)

- Regresión Lineal (VII):
 - Dividir en training/test, entrenar con training y validar con test

```
from sklearn.model_selection import train_test_split
#Separo los datos de "train" en entrenamiento y prueba para probar los algoritmos
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
#Defino el algoritmo a utilizar
lr = linear_model.LinearRegression()
#Entreno el modelo
lr.fit(X_train, y_train)
#Realizo una predicción
Y_pred = lr.predict(X_test)
#Graficamos los datos junto con el modelo
plt.scatter(X_test, y_test)
plt.plot(X_test, Y_pred, color='red', linewidth=3)
plt.title('Regresión Lineal Simple')
plt.xlabel('Número de habitaciones')
plt.ylabel('Valor medio')
plt.show()
```

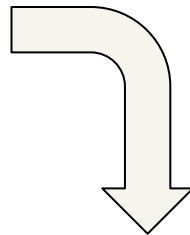
Predicción Test



Regresión (VIII)

- Regresión Lineal (VIII):
 - Ecuación del modelo y precisión

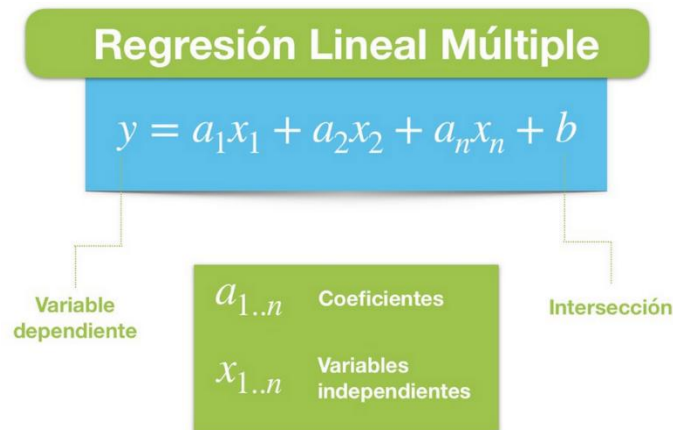
```
print('DATOS DEL MODELO REGRESIÓN LINEAL SIMPLE')
print()
print('Valor de la pendiente o coeficiente "a":')
print(lr.coef_)
print('Valor de la intersección o coeficiente "b":')
print(lr.intercept_)
print()
print('La ecuación del modelo es igual a:')
print('y = ', lr.coef_, 'x ', lr.intercept_)
print('Precisión del modelo:')
print(lr.score(X_train, y_train))
```

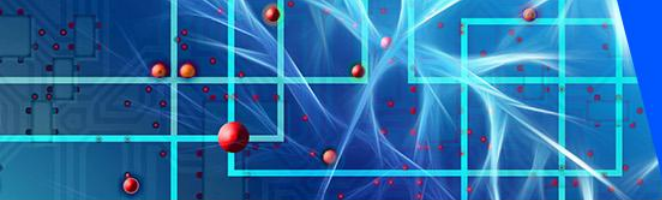


DATOS DEL MODELO REGRESIÓN LINEAL SIMPLE
Valor de la pendiente o coeficiente "a": [9.88389887]
Valor de la intersección o coeficiente "b": -39.66637453843923
La ecuación del modelo es igual a: $y = [9.88389887] x - 39.66637453843923$
Precisión del modelo: 0.5281587974403683

Regresión (IX)

- Regresión Múltiple (I):
 - La Regresión Lineal Múltiple es muy parecida a la simple, pero se manejan múltiples variables independientes que contribuyen a la variable dependiente.
 - Lo primero que hay que hacer es seleccionar las mejores variables independientes que puedan contribuir a la variable dependiente.
 - Para esto, se debe construir una matriz de correlación para todas las variables independiente e incluimos la variable dependiente.





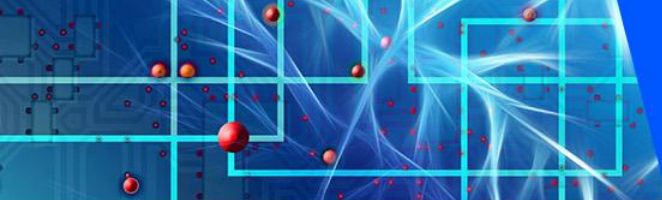
Regresión (X)

- Regresión Múltiple (II):

- Para estudiar la relación lineal existente entre dos variables continuas es necesario disponer de parámetros que permitan cuantificar dicha relación. Uno de estos parámetros es la **covarianza**, que mide el grado de variación conjunta de dos variables aleatorias.

$$\text{Covarianza muestral} = \text{Cov}(X, Y) = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{N - 1}$$

- La principal limitación de la covarianza es que, su magnitud, depende de las escalas en que se miden las variables estudiadas. Esto implica que no puede utilizarse para comparar el grado de asociación entre pares de variables en distintas escalas.
- Una forma de evitar esta limitación y poder hacer comparaciones consiste en estandarizar la covarianza, generando lo que se conoce como coeficientes de correlación.



Regresión (XI)

- Regresión Múltiple (III):
 - Algunos coeficientes de correlación:
 - La correlación de Pearson funciona bien con variables cuantitativas que tienen una distribución normal o próxima a la normal. Es más sensible a los valores extremos que las otras dos alternativas. Se define como la covarianza entre dos variables dividida por el producto de las desviaciones estándar de las dos variables.

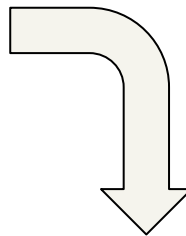
$$\rho(X, Y) = \frac{COV(X, Y)}{\sigma_X \sigma_Y}$$

- La correlación de Spearman se emplea con variables cuantitativas (continuas o discretas). En lugar de utilizar directamente el valor de cada variable, los datos son ordenados y reemplazados por su respectivo orden ranking. Es un método no paramétrico muy utilizado cuando no se satisface la condición de normalidad necesaria para aplicar la correlación de Pearson.
- La correlación de Kendall es otra alternativa no paramétrica que, al igual que la correlación de Spearman, utiliza la ordenación de las observaciones ranking. Es recomendable cuando se dispone de pocos datos y muchos de ellos ocupan la misma posición en el rango, es decir, cuando hay muchas ligaduras.

Regresión (XII)

- Regresión Múltiple (IV):
 - Matriz de correlación de Pearson (I):

```
import numpy as np
np.random.seed(10)
# generating 10 random values for each of the two variables
X = np.random.randn(10)
Y = np.random.randn(10)
# computing the correlation matrix
C = np.corrcoef(X,Y)
print(C)
```



```
[[1.         0.0247439]
 [0.0247439 1.         ]]
```

Regresión (XIII)

- Regresión Múltiple (V):
 - Matriz de correlación de Pearson (II): Variables en detección de cáncer de mama

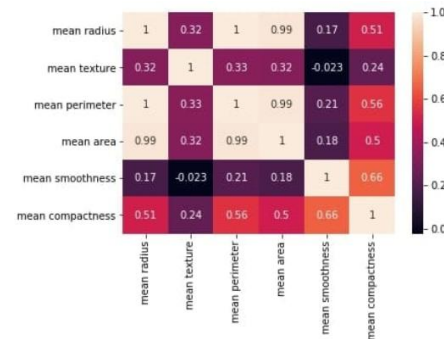
```
from sklearn.datasets import load_breast_cancer
import pandas as pd
breast_cancer = load_breast_cancer()
data = breast_cancer.data
features = breast_cancer.feature_names
df = pd.DataFrame(data, columns = features)
print(df.shape)
print(features)
```

Variables

```
(569, 30)
Index(['mean radius', 'mean texture', 'mean perimeter', 'mean area',
      'mean smoothness', 'mean compactness', 'mean concavity',
      'mean concave points', 'mean symmetry', 'mean fractal dimension',
      'radius error', 'texture error', 'perimeter error', 'area error',
      'smoothness error', 'compactness error', 'concavity error',
      'concave points error', 'symmetry error', 'fractal dimension error',
      'worst radius', 'worst texture', 'worst perimeter', 'worst area',
      'worst smoothness', 'worst compactness', 'worst concavity',
      'worst concave points', 'worst symmetry', 'worst fractal dimension'],
      dtype='object')
```

```
import seaborn as sns
import matplotlib.pyplot as plt
# taking all rows but only 6 columns
df_small = df.iloc[:, :6]
correlation_mat = df_small.corr()
sns.heatmap(correlation_mat, annot = True)
plt.show()
```

Matriz



- Valores negativos implican correlación negativa mientras que valores positivos indican correlación positiva.

Regresión (XIV)

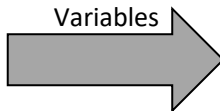
- Regresión Múltiple (VI):
 - Estimar emisiones CO2 a partir del volumen y peso de un coche

```
import pandas
from sklearn import linear_model

df =
pandas.read_csv("https://www.w3schools.com/python/cars.csv")
```

```
X = df[['Weight', 'Volume']]
y = df['CO2']
```

Variables



```
regr = linear_model.LinearRegression()
regr.fit(X, y)
```

```
#predict the CO2 emission of a car where the weight is 2300kg,
and the volume is 1300cm3:
predictedCO2 = regr.predict([[2300, 1300]])
```

```
print(predictedCO2)
```

Predicción



```
print(regr.coef_)
```

Coefficientes para X e Y

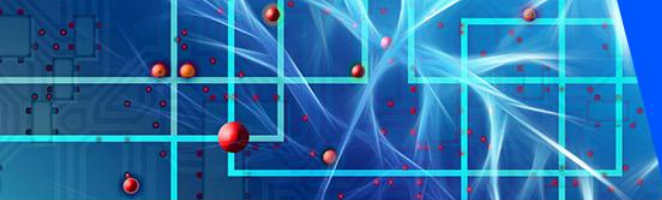


Car	Model	Volume	Weight	CO2
Toyota	Aygo	1000	790	99
Mitsubishi	Space Star	1200	1160	95
Skoda	Citigo	1000	929	95
Fiat	500	900	865	90
Mini	Cooper	1500	1140	105
VW	Up!	1000	929	105
Skoda	Fabia	1400	1109	90
Mercedes	A-Class	1500	1365	92
Ford	Fiesta	1500	1112	98
Audi	A1	1600	1150	99

https://www.w3schools.com/python/python_ml_multiple_regression.asp

[107.2087328]

[0.00755095 0.00780526]



Regresión (XV)

- Regresión No Lineal (I):
 - Ocurre cuando no podemos aproximar con una línea la solución
 - Hay distintos tipos de regresión no lineal:
 - Regresión exponencial: Muchos problemas de la naturaleza

$$Y = ae^{bX}$$

- Regresión logarítmica

$$Y = a \ln(X) + b$$

- Regresión polinomial

$$y = a + b x + c x^2 \leftarrow \text{Segundo Grado}$$

$$y = a + b x + c x^2 + d x^3 \leftarrow \text{Tercer Grado}$$

$$y = a_0 + a_1 x + a_2 x^2 + \dots a_n x^n \leftarrow \text{Ecuación general para cualquier grado}$$

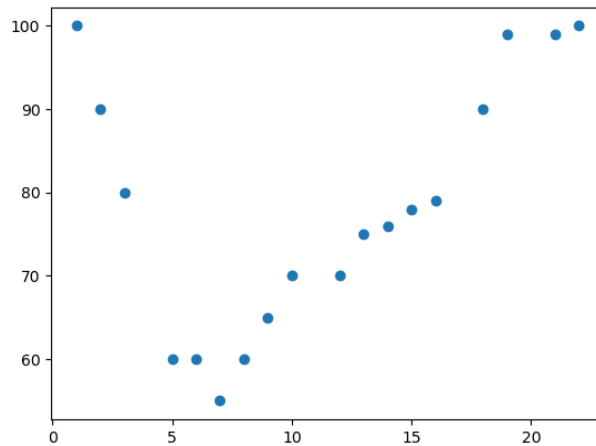
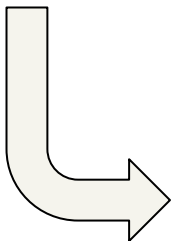
Regresión (XVI)

- Regresión No Lineal (II): Regresión Polinomial

```
import matplotlib.pyplot as plt

x = [1,2,3,5,6,7,8,9,10,12,13,14,15,16,18,19,21,22]
y = [100,90,80,60,60,55,60,65,70,70,75,76,78,79,90,99,99,100]

plt.scatter(x, y)
plt.show()
```



Regresión (XVII)

- Regresión No Lineal (III): Regresión Polinomial

```
import numpy
import matplotlib.pyplot as plt

x = [1,2,3,5,6,7,8,9,10,12,13,14,15,16,18,19,21,22]
y = [100,90,80,60,60,55,60,65,70,70,75,76,78,79,90,99,99,100]

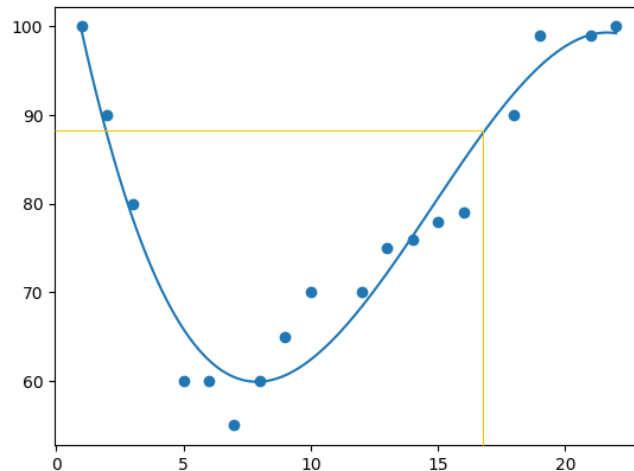
mymodel = numpy.poly1d(numpy.polyfit(x, y, 3))

myline = numpy.linspace(1, 22, 100)

plt.scatter(x, y)
plt.plot(myline, mymodel(myline))
plt.show()

speed = mymodel(17)
print(speed)
```

Predicción



Regresión (XVIII)

- Regresión No Lineal (IV): Regresión Polinomial
 - Es importante conocer la relación entre los valores de los ejes x e y, si no hay relación la regresión polinómica no puede utilizarse para predecir nada.
 - La relación se mide con un valor llamado R-cuadrado.
 - El valor R-cuadrado oscila entre 0 y 1, donde 0 significa que no hay relación, y 1 significa que hay un 100% de relación

```
import numpy
from sklearn.metrics import r2_score

x = [1,2,3,5,6,7,8,9,10,12,13,14,15,16,18,19,21,22]
y = [100,90,80,60,60,55,60,65,70,70,75,76,78,79,90,99,99,100]

mymodel = numpy.poly1d(numpy.polyfit(x, y, 3))
print(r2_score(y, mymodel(x)))
```



0,94 (Buena
regresión)

Regresión (XIX)

La fórmula para calcular R^2 es:

$$R^2 = 1 - \frac{\text{SCR}}{\text{SCT}}$$

Donde:

- SCR es la Suma de Cuadrados de los Residuos (o error), que representa la diferencia entre los valores observados y los valores ajustados por el modelo.
- SCT es la Suma Total de Cuadrados, que representa la variación total de los datos con respecto a la media.

Las fórmulas para SCR y SCT son:

$$\text{SCR} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\text{SCT} = \sum_{i=1}^n (y_i - \bar{y})^2$$

Donde:

- y_i es el valor observado.
- $\hat{y}_i$ es el valor predicho por el modelo.
- $\bar{y}$ es la media de los valores observados.
- n es el número de observaciones.

Regresión (XX)

- Regresión No Lineal (V): Regresión Polinomial
 - Ejemplo de mal ajuste

```
import numpy
import matplotlib.pyplot as plt
from sklearn.metrics import r2_score
```

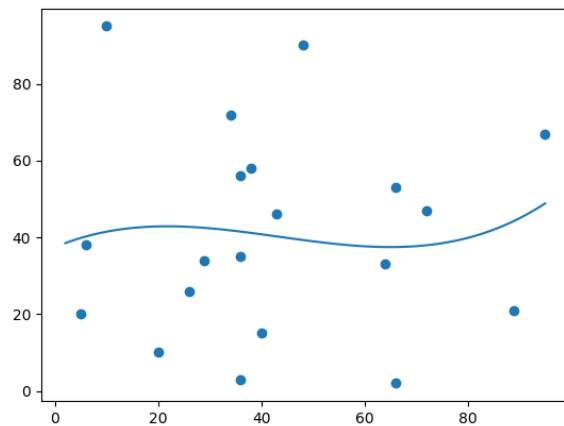
```
x = [89,43,36,36,95,10,66,34,38,20,26,29,48,64,6,5,36,66,72,40]
y = [21,46,3,35,67,95,53,72,58,10,26,34,90,33,38,20,56,2,47,15]
```

```
mymodel = numpy.poly1d(numpy.polyfit(x, y, 3))
```

```
myline = numpy.linspace(2, 95, 100)
```

```
plt.scatter(x, y)
plt.plot(myline, mymodel(myline))
plt.show()
```

```
print(r2_score(y, mymodel(x)))
```



https://www.w3schools.com/python/python_ml_polynomial_regression.asp



0,00995 (Mala
regresión)

Regresión (XXI)

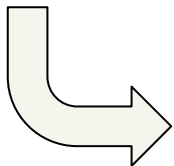
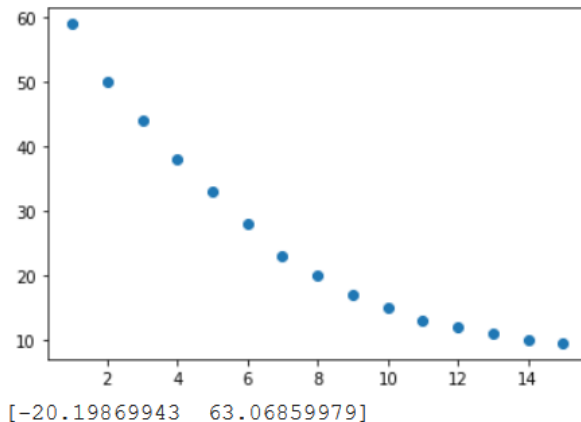
- Regresión No Lineal (VI): Regresión Logarítmica

```
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(1, 16, 1)
y = [59, 50, 44, 38, 33, 28, 23, 20, 17, 15, 13, 12, 11, 10, 9.5]

plt.scatter(x, y)
plt.show()

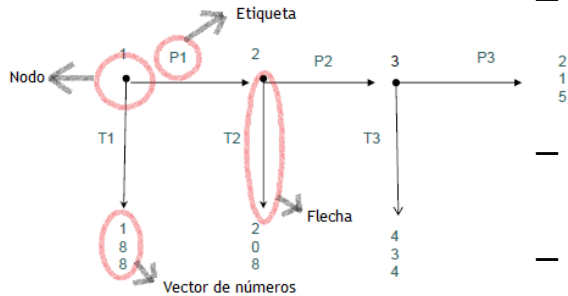
#ajuste el modelo
fit = np.polyfit(np.log(x), y, 1)
#ver la salida del modelo
print(fit)
```



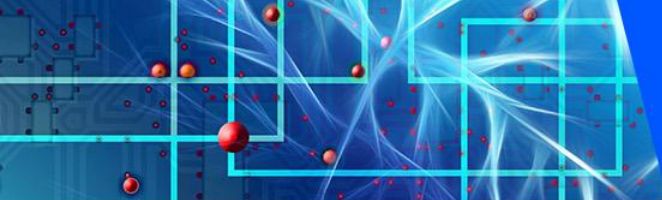
$$y = 63.0686 - 20.1987 * \ln(x)$$

Árbol de decisión (I)

- Los árboles de decisión tienen gran utilidad en muchos campos, incluyendo la teoría de juegos.
- Elementos:



- **Nodo:** momento en el que se ha de tomar una decisión de entre varias posibles. Un árbol con muchos nodos es complicado de dibujar y analizar manualmente.
- **Vector:** Etiqueta numérica final a la que se llega en función de las diversas posibilidades que se tienen.
- **Flechas:** Uniones entre un nodo y otro y representan cada acción distinta.
- **Etiquetas:** se encuentran en cada nodo y cada flecha y dan nombre a cada acción.

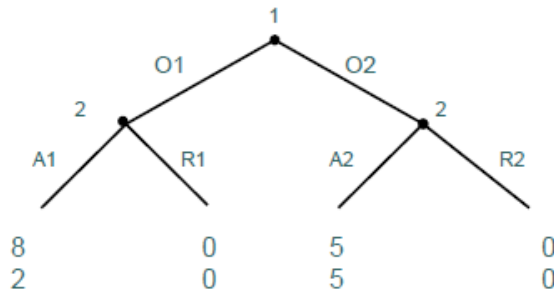


Árbol de decisión (II)

- Conceptos importantes:
 - Costo:
 - Determinar el valor de una determinada propiedad (atributo) exhibida por el objeto
 - Costo de clasificación errónea al decidir que el objeto pertenece a la clase X cuando su clase real es Y .
 - Sobreajuste (Overfitting): Pocos datos de entrenamiento o diseño incorrecto del árbol.
 - *Al tomar un espacio de hipótesis H , se dice que una hipótesis $h \in H$ sobreajusta un conjunto de entrenamiento C si existe alguna hipótesis alternativa $h' \in H$ tal que h clasifica mejor que h' los elementos del conjunto de entrenamiento, pero h' clasifica mejor que h el conjunto completo de posibles instancias.*
 - Poda (Prunning): Eliminar una rama de un nodo transformándolo en una hoja (terminal), asignándole la clasificación más común de los ejemplos de entrenamiento considerados en ese nodo.
 - Validación cruzada: Construir un árbol con la mayoría de los datos y luego usar la parte restante de los datos para probar la precisión del árbol.
- Hay un único camino para llegar del nodo inicial a cada uno de los nodos del juego. No hay varias formas de llegar a la misma solución final, las decisiones son excluyentes.

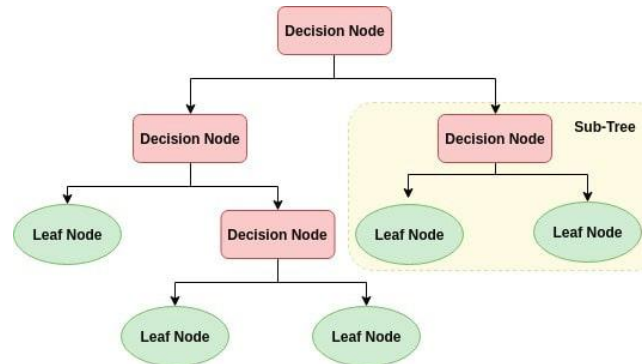
Árbol de decisión (III)

- Ejemplo:
 - En este árbol de decisión podemos observar cómo hay cuatro posibles soluciones finales. En él se cumplen las normas antes descritas (nodo inicial, una única flecha por nodo y un único camino para llegar a cada nodo final) y tiene todos los elementos antes descritos (nodos, vectores de números, flechas y etiquetas). La imagen representa un árbol de decisión conformado por dos jugadores, en él se pueden observar las opciones que el juego cuenta con dos jugadoras. La primera decisión la ha de tomar la jugadora 1, quien debe decidir entre O1 y O2, en este punto será la jugadora 2 quien decida. Si la Jugadora 1 ha elegido O1, tendrá que decidir entre A1 y R1, A1 le producirá una utilidad de 2, y R1 de 0, a su vez A1 le reportará una utilidad de 8 a la jugadora 1 y R1 de 0. En cambio, si la jugadora 1 elige O2, la jugadora 2 deberá elegir entre A2 y R2, la primera opción le reportará una utilidad de 5 a ella y a la otra jugadora, R2 reportará una utilidad de 0 a las dos. Con los adecuados métodos de inducción existentes se podría resolver este árbol de decisión sin mucha complicación.



Árbol de decisión (IV)

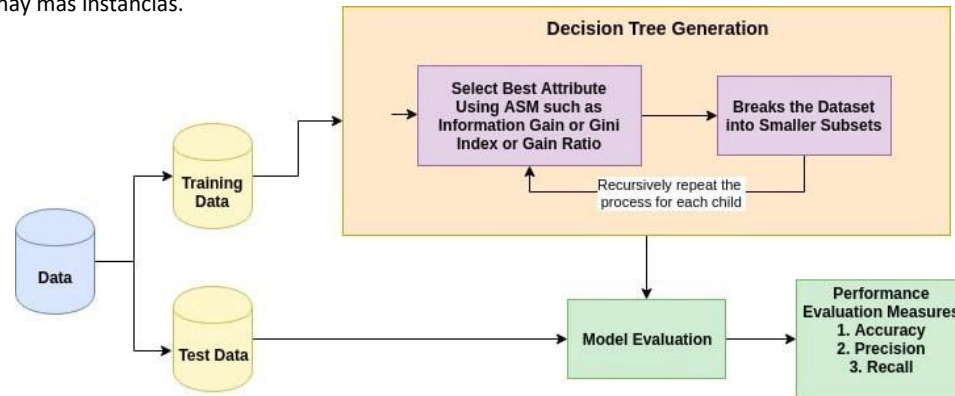
- Algoritmo de construcción (I):
- Los árboles de decisión clasifican los ejemplos clasificándolos por el árbol desde la raíz hasta algún nodo hoja, con el nodo hoja proporcionando la clasificación al ejemplo (*Enfoque de arriba hacia abajo*).
- Cada nodo en el árbol actúa como un caso de prueba para algún atributo, y cada flecha que desciende de ese nodo corresponde a una de las posibles respuestas al caso de prueba. Este proceso es recursivo y se repite para cada subárbol enraizado en los nuevos nodos.



Árbol de decisión (V)

- **Algoritmo de construcción (II):**

1. Seleccione el mejor atributo utilizando Medidas de selección de atributos (ASM) para dividir los registros. La medida de selección de atributos (ASM) es una heurística para seleccionar el criterio de división que divide los datos de la mejor manera posible.
2. Haga que ese atributo sea un nodo de decisión y divida el conjunto de datos en subconjuntos más pequeños. recursivamente para cada hijo hasta que una de las condiciones coincida:
 - Todas las tuplas pertenecen al mismo valor de atributo.
 - No quedan más atributos.
 - No hay más instancias.

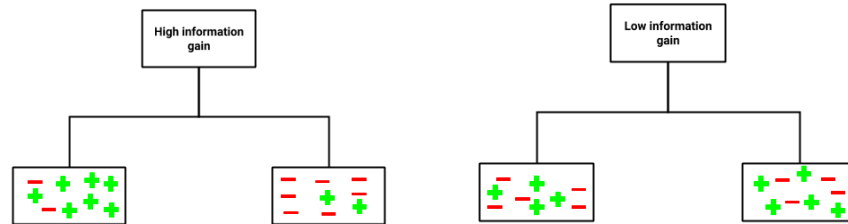


Árbol de decisión (VI)

- Algoritmo de construcción (III)

- ASM:

- Ganancia de Información: Un atributo con alta ganancia de información divide los datos en grupos con un número desigual de positivos y negativos y, como resultado, ayuda a separarlos entre sí.



- Para calcular la ganancia de información, debemos conocer la entropía (Shannon) que mide la impureza de un conjunto de entrada. La entropía es 1 cuando la muestra contiene el mismo número de ejemplos positivos y negativos. Si la muestra contiene un número desigual de ejemplos positivos y negativos, la entropía está entre 0 y 1.

Árbol de decisión (IX)

- Algoritmo de construcción (IV)

- ASM. Ganancia de Información

- Ejemplo: Tenemos una muestra de 30 estudiantes con tres variables Sexo (Niño / Niña), Clase (IX / X) y Altura (5 a 6 pies). 15 de estos 30 juegan al cricket en el tiempo libre. Queremos crear un modelo para predecir quién jugará al cricket durante un período de ocio.

Entropía (nodo principal) = $-(15/30) \log_2 (15/30) - (15/30) \log_2 (15/30) = 1$ (Es un nodo impuro. Altamente mezclado)

Split on Gender

Students = 30
Play Cricket = 15 (50%)

Female



Students = 10
Play Cricket = 2 (20%)

Male



Students = 20
Play Cricket = 13 (65%)

Split on Class



Class IX



Students = 14
Play Cricket = 6 (43%)

Class X



Students = 16
Play Cricket = 9 (56%)

<https://sitiobigdata.com/2019/12/14/arbol-de-decision-en-machine-learning-parte-1/#>

Dividir en género:

Entropía para nodo femenino = $-(2/10) \log_2 (2/10) - (8/10) \log_2 (8/10) = 0,72$

Entropía para el nodo masculino = $-(13/20) \log_2 (13/20) - (7/20) \log_2 (7/20) = 0,93$

Entropía por división Género = $(10/30) * 0,72 + (20/30) * 0,93 = 0,86$

Ganancia de información por división por género = $1 - 0,86 = 0,14$

Dividir en clase:

Entropía para nodo de clase IX = $-(6/14) \log_2 (6/14) - (8/14) \log_2 (8/14) = 0,99$

Entropía para clase Nodo X = $-(9/16) \log_2 (9/16) - (7/16) \log_2 (7/16) = 0,99$

Entropía para división Clase = $(14/30) * 0,99 + (16/30) * 0,99 = 0,99$

Ganancia de información para división por Class = $1 - 0,99 = 0,01$

La división en género es la mejor al tener mayor ganancia de información

Árbol de decisión (X)

Ganancia de Gini (Índice de Gini)

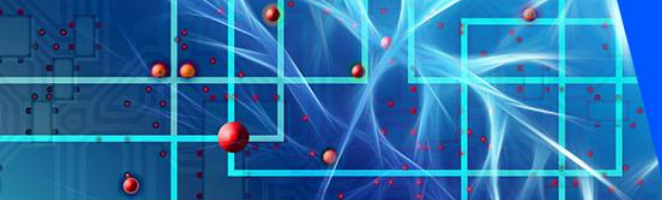
Al igual que con la ganancia de información, la ganancia de Gini se utiliza para medir la reducción del índice de Gini cuando se divide un conjunto de datos según un atributo específico. La fórmula para la ganancia de Gini $\Delta Gini(A)$ al dividir el conjunto de datos S según un atributo A es:

$$\Delta Gini(A) = G(S) - \sum_{v \in \text{Valores}(A)} \frac{|S_v|}{|S|} G(S_v)$$

Donde:

- $G(S)$ es el índice de Gini del conjunto original S .
- $\text{Valores}(A)$ es el conjunto de valores posibles del atributo A .
- S_v es el subconjunto de S donde el atributo A toma el valor v .
- $|S_v|$ es el número de elementos en el subconjunto S_v .
- $|S|$ es el número de elementos en el conjunto original S .
- $G(S_v)$ es el índice de Gini del subconjunto S_v .

Un índice de Gini de 0 indica que el conjunto de datos es perfectamente puro (todos los elementos pertenecen a la misma clase), mientras que un valor mayor indica mayor impureza. La ganancia de Gini mide cuánto mejora (disminuye) el índice de Gini cuando el conjunto de datos se divide según un atributo.

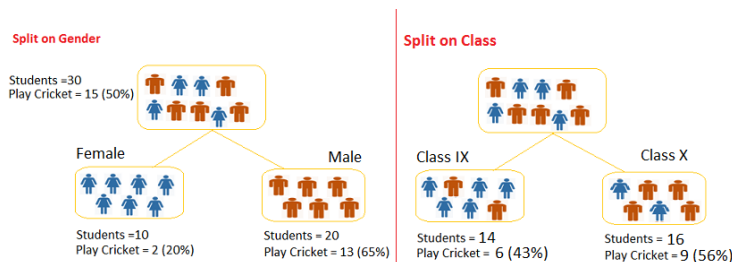


Árbol de decisión (XI)

- Algoritmo de construcción (V)
 - ASM.
 - GINI:
 - GINI dice: “si seleccionamos dos elementos de una población al azar, entonces deben ser de la misma clase y la probabilidad de esto es 1 si la población es pura.”
 - Funciona con la variable objetivo categórica “Éxito” o “Fracaso” .
 - Realiza solo divisiones binarias
 - Cuanto mayor sea el valor de GINI, mayor será la homogeneidad.
 - CART (árbol de clasificación y regresión) utilizan el método GINI para crear divisiones binarias.
 - Pasos para calcular GINI para una división:
 - » Calcule GINI para subnodos, usando la suma de la fórmula del cuadrado de probabilidad de éxito y fracaso ($p^2 + q^2$).
 - » Calcule GINI para la división usando la puntuación ponderada de GINI de cada nodo de esa división

Árbol de decisión (XII)

- Algoritmo de construcción (VI)
 - ASM.
 - GINI: Ejemplo



- A veces encontramos "Impureza de GINI":
Impureza de Gini = 1 - Gini

<https://sitiobigdata.com/2019/12/14/arbol-de-decision-en-machine-learning-parte-1/#>

División en género:

Calcular, Gini para el subnodo Hembra = $(0.2) * (0.2) + (0.8) * (0.8) = 0.68$

Gini para sub-nodo Macho = $(0.65) * (0.65) + (0.35) * (0.35) = 0.55$

Calcular Gini ponderado para Split Gender = $(10/30) * 0.68 + (20/30) * 0.55 = \mathbf{0.59}$

División en Clase:

Gini para sub-nodo Clase IX = $(0.43) * (0.43) + (0.57) * (0.57) = 0.51$

Gini para sub-nodo Clase X = $(0.56) * (0.56) + (0.44) * (0.44) = 0.51$

Calcular Gini ponderado para Clase dividida = $(14/30) * 0.51 + (16/30) * 0.51 = \mathbf{0.51}$

La puntuación de Gini para *Dividir en género* es mayor que *Dividir en Clase*, por lo tanto, la división de nodos tendrá lugar en Género.

Árbol de decisión (XIII)

- Clasificación multi-clase:
Planta Iris

- Se busca clasificar el tipo de planta Iris a partir de un conjunto de datos que contiene 3 clases de 50 instancias cada una, donde cada clase se refiere a un tipo de planta de iris. Una clase es linealmente separable de las otras dos; estas últimas no son linealmente separables entre sí.
- A partir de un conjunto de los datos de una nueva planta se busca predecir la clase de planta de iris.
- Los datos del dataset son:
 1. Longitud del sépalo en cm
 2. Anchura del sépalo en cm
 3. Longitud del pétalo en cm
 4. Anchura del pétalo en cm
 5. Clase: Setosa/Versicolor/Virginica



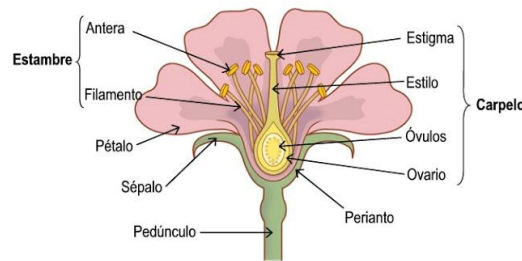
Iris Setosa

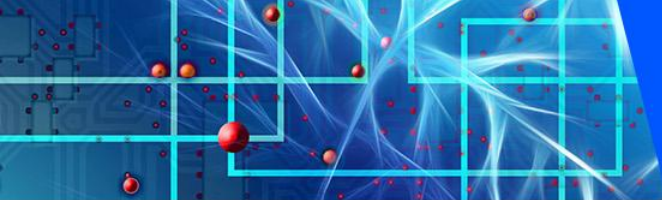


Iris Versicolor



Iris Virginica





Árbol de decisión (XIV)

- Se puede realizar práctica en Google Colab:
 - <https://colab.research.google.com/>
- Clasificación de las flores de la planta Iris y representación de su árbol de decisión

```
from sklearn.datasets import load_iris
from sklearn import tree
X, y = load_iris(return_X_y=True)
clf = tree.DecisionTreeClassifier()
clf = clf.fit(X, y)
tree.plot_tree(clf)

# make a prediction
from numpy import argmax
row = [5.1, 3.5, 1.4, 0.2]
yhat = clf.predict_proba([row])
print('Predicted: %s (class=%d)' % (yhat, argmax(yhat)))
```

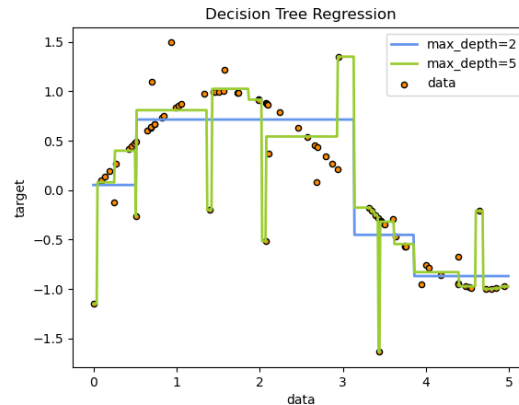
Árbol de decisión (XV)



Árbol de decisión (XVI)

- Otros problemas a parte de clasificación:
 - Regresión: En vez de devolver una clase (a partir de la mayor probabilidad), devuelven un valor flotante concreto.

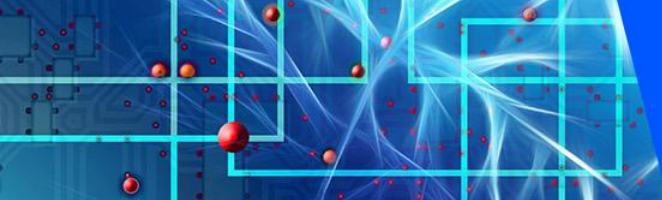
```
from sklearn import tree
X = [[0, 0], [2, 2]]
y = [0.5, 2.5]
clf = tree.DecisionTreeRegressor()
clf = clf.fit(X, y)
clf.predict([[3, 3]])
```



Árbol de decisión (XVII)

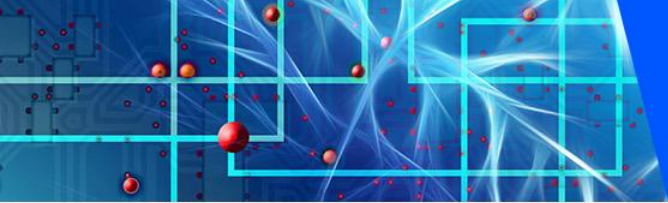
- Otros problemas a parte de clasificación:
 - Multi-output problems: En vez de una única salida, se esperan múltiples salidas.
 - Se puede implementar con n modelos diferentes (n árboles)
 - Por la alta correlación de algunas variables, se puede utilizar un único modelo
 - Ejemplo: Reconstrucción de una cara a partir de la parte superior. Las n salidas corresponden a los píxeles de la parte inferior de la cara.





Árbol de decisión (XVIII)

- Complejidad computacional:
 - Entrenamiento de un modelo: $O(n_{samples}n_{features} \log(n_{samples}))$
 - Tiempo de consulta: $O(\log(n_{samples}))$.
- Minimal cost-complexity pruning es un algoritmo utilizado para podar un árbol con el fin de evitar el overfitting

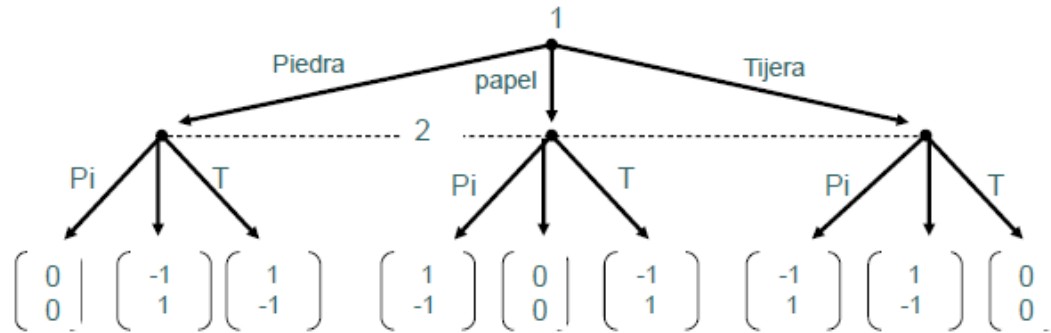


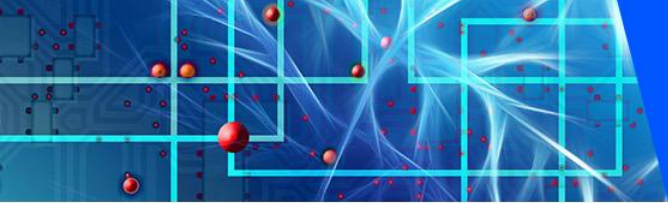
Árbol de decisión (XIX)

- Algoritmos desarrollados:
 - ID3 (Iterative Dichotomiser 3) fue desarrollado en 1986 por Ross Quinlan. El algoritmo crea un árbol multidireccional, encontrando para cada nodo (es decir, de forma codiciosa) la característica categórica que producirá la mayor ganancia de información para los objetivos categóricos. Los árboles crecen hasta su tamaño máximo y, a continuación, se suele aplicar un paso de poda para mejorar la capacidad del árbol de generalizar a los datos no vistos.
 - C4.5 es el sucesor de ID3 y elimina la restricción de que las características sean categóricas definiendo dinámicamente un atributo discreto (basado en variables numéricas) que divide el valor del atributo continuo en un conjunto discreto de intervalos. C4.5 convierte los árboles entrenados (es decir, la salida del algoritmo ID3) en conjuntos de reglas if-then. A continuación, se evalúa la precisión de cada regla para determinar el orden en que deben aplicarse. La poda se realiza eliminando la precondition de una regla si la precisión de esta mejora sin ella.
 - C5.0 es la última versión de Quinlan con licencia de propiedad. Utiliza menos memoria y construye conjuntos de reglas más pequeños que C4.5, pero es más preciso.
 - CART (Classification and Regression Trees) es muy similar a C4.5, pero difiere en que admite variables objetivo numéricas (regresión) y no computa conjuntos de reglas. **CART construye árboles binarios** utilizando la característica y el umbral que producen la mayor ganancia de información en cada nodo.

Árbol de decisión (XX)

- Actividad: Intentad desarrollar en Google Colab el juego Piedra-Papel-Tijera:



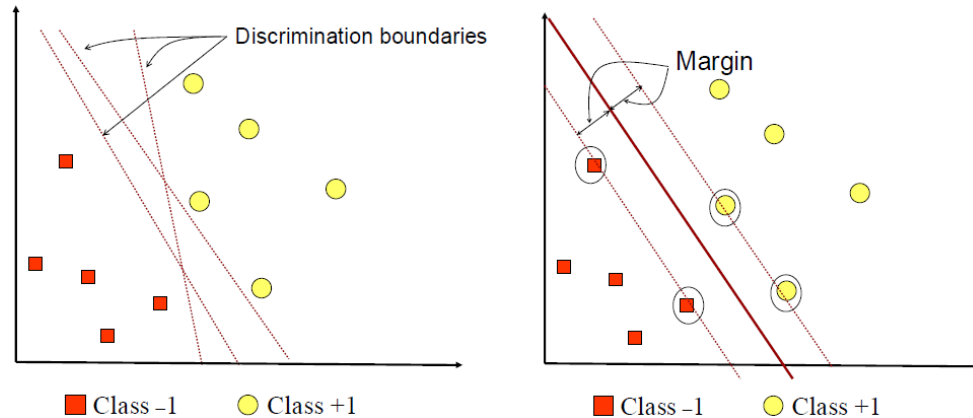


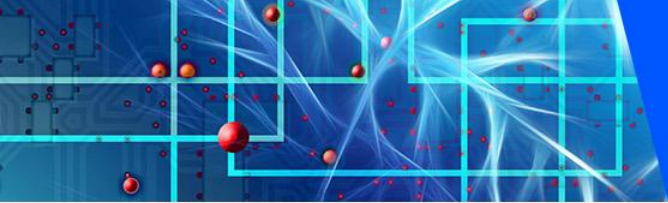
SVM (I)

- Una máquina de vectores de apoyo (SVM) es un algoritmo de aprendizaje supervisado que se utiliza para muchos problemas de clasificación y regresión, como las aplicaciones médicas de procesamiento de señales, el procesamiento del lenguaje natural y el reconocimiento de voz e imágenes.
- El objetivo del algoritmo SVM es encontrar un hiperplano que, en el mejor grado posible, separe los puntos de datos de una clase de los de otra clase. "Mejor" se define como el hiperplano con el mayor margen entre las dos clases.
- Sólo para los problemas linealmente separables puede el algoritmo encontrar tal hiperplano, para la mayoría de los problemas prácticos el algoritmo maximiza el margen permitiendo un pequeño número de clasificaciones erróneas.

SVM (II)

- Los vectores de soporte se refieren a un subconjunto de las observaciones de entrenamiento que identifican la ubicación del hiperplano de separación. El algoritmo estándar de la SVM está formulado para problemas de clasificación binarios, y los problemas multiclase suelen reducirse a una serie de binarios.





SVM (III)

- Las máquinas de vectores de apoyo pertenecen a una clase de algoritmos de aprendizaje automático que utilizan núcleos de operación (kernel), con los que las características pueden transformarse mediante una función kernel.
- Las funciones kernel mapean los datos a un espacio diferente, a menudo de mayor dimensión, con la expectativa de que las clases sean más fáciles de separar después de esta transformación, simplificando potencialmente unos complejos límites de decisión no lineales a unos lineales en el espacio de características mapeado de mayor dimensión (kernel trick o truco del núcleo). En este proceso, los datos no tienen que ser transformados explícitamente, lo que sería computacionalmente caro (esto se hace matemáticamente gracias a ciertas propiedades del álgebra).

SVM (IV)

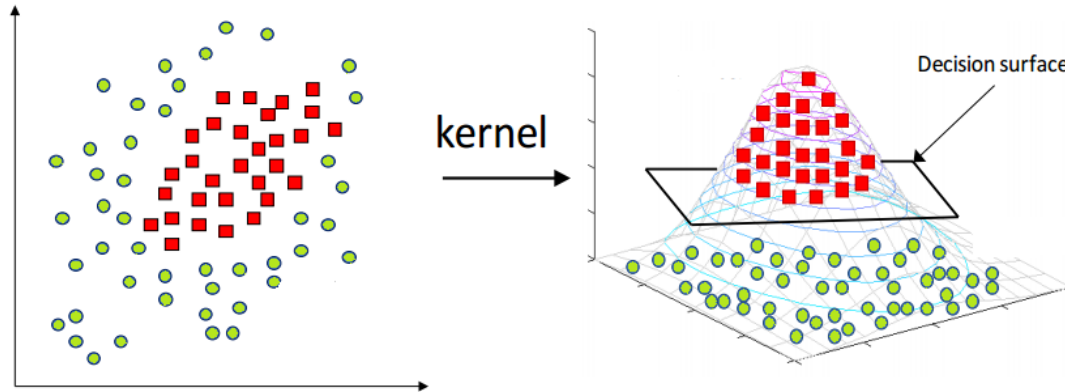
- Ejemplos de kernels:

Type of SVM	Mercer Kernel	Description
Gaussian or Radial Basis Function (RBF)	$K(x_1, x_2) = \exp\left(-\frac{\ x_1 - x_2\ ^2}{2\sigma^2}\right)$	One class learning. σ is the width of the kernel
Linear	$K(x_1, x_2) = x_1^T x_2$	Two class learning.
Polynomial	$K(x_1, x_2) = (x_1^T x_2 + 1)^\rho$	ρ is the order of the polynomial
Sigmoid	$K(x_1, x_2) = \tanh(\beta_0 x_1^T x_2 + \beta_1)$	It is a mercer kernel for certain β_0 and β_1 values only

<http://es.mathworks.com/discovery/support-vector-machine.html>

SVM (V)

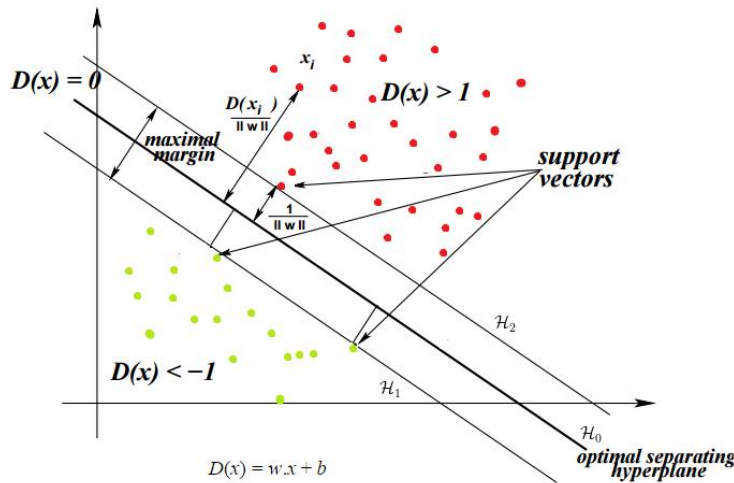
- Aplicación de un Kernel: Transformamos un conjunto de datos original en un conjunto de datos separable linealmente



<http://blog-c7ff.kxcdn.com/blog/wp-content/uploads/2017/02/kernel.png>

SVM (VI)

- Separación lineal de dos conjuntos de datos: Debemos encontrar un conjunto de pesos, w , que permitan separar dos grupos de datos aproximando uno de ellos al valor $+1$ y otro al valor -1



- Para maximizar el margen, utilizamos optimización. Si el margen es

$$m = \frac{2}{\|\vec{w}\|}$$

- Para maximizar m , tenemos que minimizar

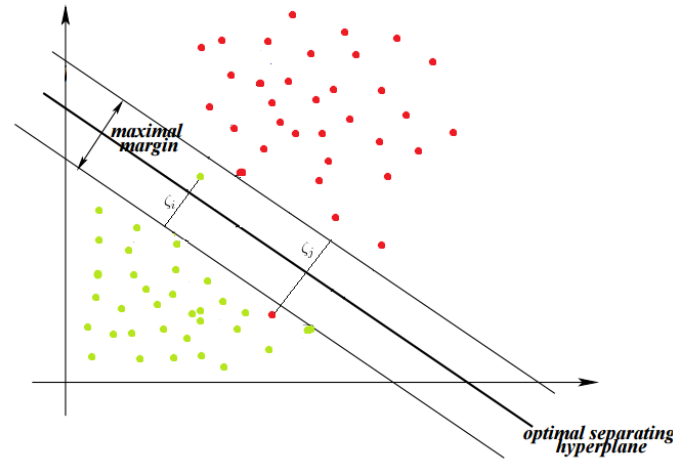
$$\text{Minimize in } (\vec{w}, b) \left\{ \frac{\|\vec{w}\|^2}{2} \right\}$$

subject to $y_i (\vec{w} \cdot \vec{x} + b) \geq 1$ for any $i = 1, \dots, n$

- donde y_i vale $1/-1$ dependiendo de la clase.

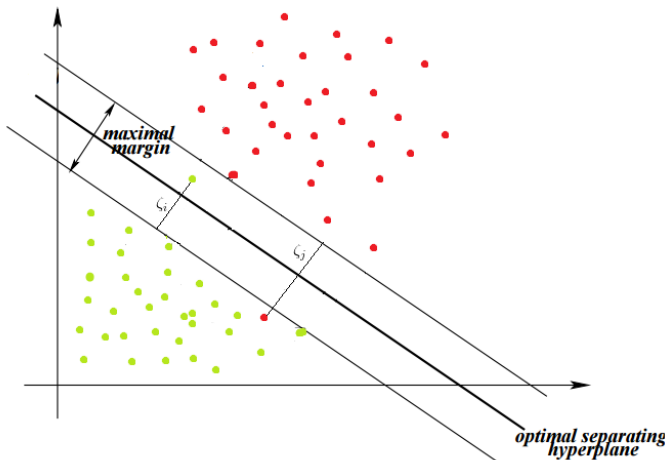
SVM (VII)

- En la vida real no siempre los problemas son separables linealmente completamente. Habrá muestras que queden en el medio.
- En este escenario, el algoritmo intenta mantener la variable de holgura a cero mientras maximiza el margen. Sin embargo, minimiza la suma de las distancias de la clasificación errónea desde los hiperplanos de margen y no el número de clasificaciones erróneas.



<http://blog-c7ff.kxcdn.com/blog/wp-content/uploads/2017/02/constraints.png>

SVM (VIII)



<http://blog-c7ff.kxcdn.com/blog/wp-content/uploads/2017/02/constraints.png>

- Las restricciones cambian ahora a:

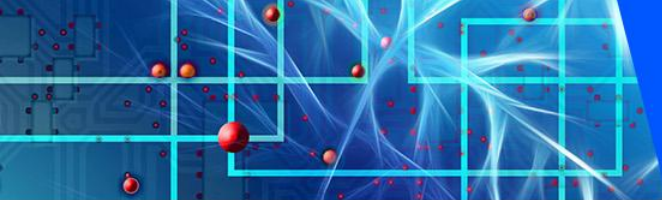
$$y_i(\vec{w} \cdot \vec{x}_i + b) \geq 1 - \zeta_i \text{ for all } 1 \leq i \leq n, \zeta_i \geq 0$$

- Para maximizar m , tenemos que minimizar

$$\text{Minimize in } (\vec{w}, b) \left\{ \frac{\|\vec{w}\|^2}{2} + C \sum_i \zeta_i \right.$$

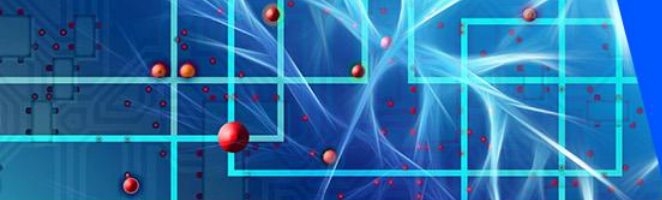
$$\left. \text{subject to } y_i(\vec{w} \cdot \vec{x} + b) \geq 1 - \zeta_i \text{ for any } i = 1, \dots, n \right.$$

- el parámetro C es el parámetro de regularización que controla la compensación entre la penalización de la variable de holgura



SVM (IX)

- Parámetro de regularización:
 - Una C pequeña permite ignorar fácilmente las restricciones, lo que conduce a un margen grande.
 - C grande permite que las restricciones sean difíciles de ignorar, lo que lleva a un margen pequeño.
 - En el caso de $C=\infty$, se aplican todas las restricciones.
- La forma más sencilla de separar dos clases de datos es una línea en el caso de los datos 2D y un plano en el caso de los datos 3D. Pero no siempre es posible utilizar líneas o planos y se requiere una región no lineal para separar estas clases. Las máquinas de vectores de apoyo manejan estas situaciones utilizando una función de núcleo que mapea los datos a un espacio diferente en el que se puede utilizar un hiperplano lineal para separar las clases. Esto se conoce como el truco del kernel, en el que la función del kernel transforma los datos en un espacio de características de mayor dimensión para que sea posible una separación lineal.



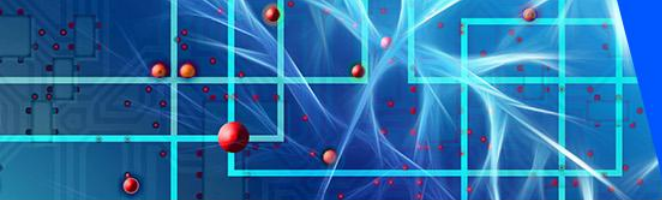
SVM (X)

- Si ϕ es la función kernel que mapea x_i a $\phi(x_i)$, entonces las restricciones cambian a:

$$y_i(\vec{w} \cdot \phi(x_i) + b) \geq 1 - \zeta_i \text{ for all } 1 \leq i \leq n, \zeta_i \geq 0$$

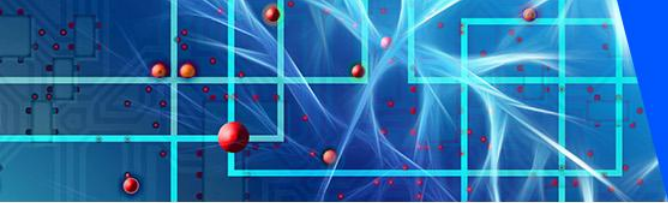
$$\text{Minimize in } (\vec{w}, b) \left\{ \frac{\|\vec{w}\|^2}{2} + C \sum_i \zeta_i \right\}$$

$$\text{subject to } y_i(\vec{w} \cdot \phi(x_i) + b) \geq 1 - \zeta_i \text{ for all } 1 \leq i \leq n, \zeta_i \geq 0$$



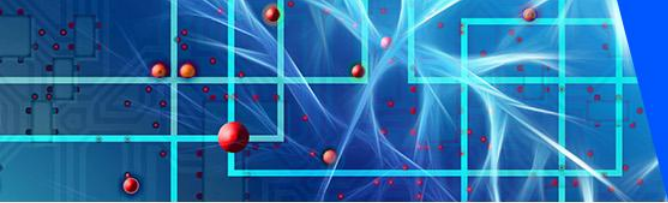
SVM (XI)

- Ventajas y desventajas de las máquinas de vectores de apoyo.
 - Cada algoritmo de clasificación tiene sus propias ventajas y desventajas que entran en juego según el conjunto de datos que se analice. Algunas de las **ventajas (I)** de las SVM son las siguientes:
 - La propia naturaleza del método de Optimización Convexa asegura la optimalidad garantizada.
 - Se garantiza que la solución es un mínimo global y no un mínimo local.
 - La SVM es un algoritmo adecuado para datos separables tanto lineal como no linealmente (utilizando el truco del kernel). Lo único que hay que hacer es idear el término de regularización, C .
 - Las SVM funcionan bien tanto en espacios de datos pequeños como en los de alta dimensión. Funciona eficazmente en conjuntos de datos de alta dimensión debido al hecho de que la complejidad del conjunto de datos de entrenamiento en la SVM se caracteriza generalmente por el número de vectores de soporte y no por la dimensionalidad. Incluso si se eliminan todos los demás ejemplos de entrenamiento y se repite el entrenamiento, obtendremos el mismo hiperplano de separación óptimo.



SVM (XII)

- Ventajas y desventajas de las máquinas de vectores de apoyo.
 - Ventajas (II) :
 - Las SVM pueden funcionar eficazmente en conjuntos de datos de entrenamiento más pequeños, ya que no dependen de la totalidad de los datos.
 - Desventajas:
 - No son adecuadas para conjuntos de datos más grandes porque el tiempo de entrenamiento con las SVM puede ser alto y mucho más intensivo desde el punto de vista computacional.
 - Son menos eficaces en conjuntos de datos más ruidosos que tienen clases superpuestas.

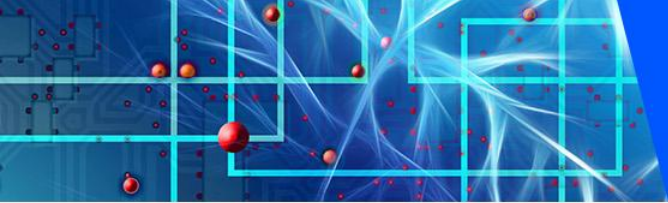


SVM (XIII)

- SVM en Python (I)
 - La biblioteca más utilizada para implementar algoritmos de aprendizaje automático en Python es scikit-learn. La clase utilizada para la clasificación SVM en scikit-learn es `svm.SVC()`

`sklearn.svm.SVC (C=1.0, kernel='rbf', degree=3, gamma='auto')`

- Los parámetros son los siguientes
 - C: Es el parámetro de regularización, C, del término de error.
 - kernel: Especifica el tipo de kernel a utilizar en el algoritmo. Puede ser 'lineal', 'poli', 'rbf', 'sigmoide', 'precomputado', o un callable. El valor por defecto es 'rbf'.
 - grado: Es el grado de la función del núcleo polinómico ('poly') y es ignorado por todos los demás núcleos. El valor por defecto es 3.
 - gamma: Es el coeficiente del kernel para 'rbf', 'poly', y 'sigmoid'. Si gamma es 'auto', entonces se utilizará $1/n_{\text{figuras}}$ en su lugar.



SVM (XIV)

- SVM en Python (II)
 - Se puede afinar la SVM cambiando los parámetros C , γ y la función del kernel. La función para afinar los parámetros disponibles en scikit-learn se llama `gridSearchCV()`.

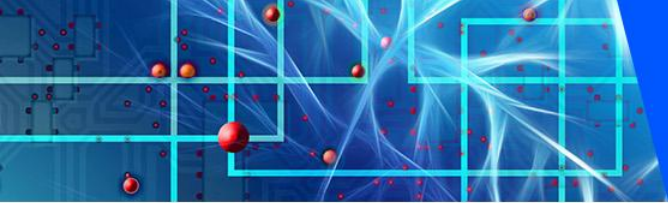
```
sklearn.model_selection.GridSearchCV(estimador, param_grid)
```

- Los parámetros de esta función se definen como:
 - `estimador`: Es el objeto estimador que en nuestro caso es `svm.SVC()`.
 - `param_grid`: Es el diccionario o lista con los nombres de los parámetros (cadena) como claves y las listas de ajustes de los parámetros a probar como valores.

SVM (XV)

- Clasificación de las flores de la planta Iris

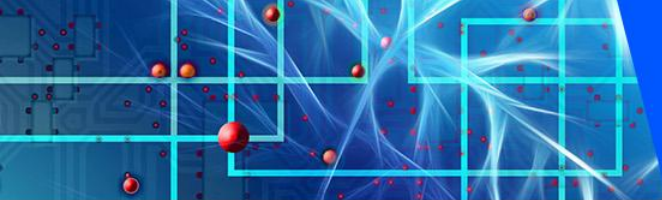
```
from sklearn.datasets import load_iris
from sklearn import tree
from sklearn import svm
from numpy import argmax
# Cargar el dataset Iris
X, y = load_iris(return_X_y=True)
# Definir el modelo SVM
clf = svm.SVC (C=1.0, kernel='rbf', degree=3, gamma='auto')
# Entrenar el modelo SVM
clf = clf.fit(X, y)
# Hacer una predicción
row = [5.1, 3.5, 1.4, 0.2]
yhat = clf.predict([row])
print("Predicted class= %d" % (yhat[0]))
```



SVM (XVI)

- Clasificación de las flores de la planta Iris (con grid-estimator)

```
from sklearn import svm, datasets
from sklearn.model_selection import GridSearchCV
iris = datasets.load_iris()
parameters = {'kernel':('linear', 'rbf'), 'C':[1, 10]}
svc = svm.SVC()
clf = GridSearchCV(svc, parameters)
clf.fit(iris.data, iris.target)
print(clf.best_params_)
# Analizar los resultados
print(clf.cv_results_)
# Hacer una predicción
row = [5.1, 3.5, 1.4, 0.2]
yhat = clf.predict([row])
print("Predicted class= %d" % (yhat[0]))
```



SVM (XVII)

- Cross-Validation (I):
 - Aprender los parámetros de una función de predicción y probarla sobre los mismos datos es un error metodológico: un modelo que se limitara a repetir las etiquetas de las muestras que acaba de ver tendría una puntuación perfecta, pero no lograría predecir nada útil sobre datos aún no vistos. Esta situación se llama sobreajuste (overfitting). Para evitarlo, es una práctica común cuando se realiza un experimento de aprendizaje automático (supervisado) mantener una parte de los datos disponibles como conjunto de prueba (test).
 - En scikit-learn una división aleatoria en conjuntos de training y test puede ser rápidamente calculada con la función de ayuda `train_test_split`.

SVM (XVIII)

- Cross-Validation (II):
 - Ejemplo de entrenamiento de IRIS a partir de training y evaluación de los resultados con el test:

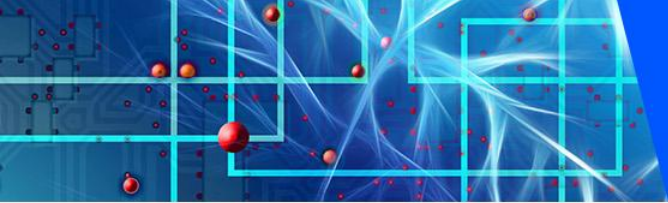
```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn import datasets
from sklearn import svm

X, y = datasets.load_iris(return_X_y=True)
X.shape, y.shape

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.4, random_state=0)

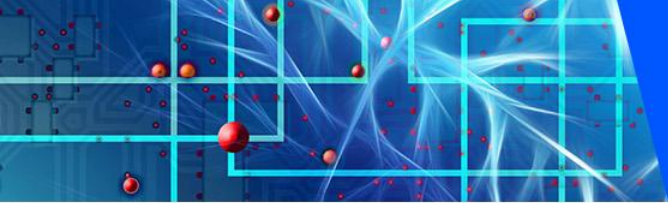
X_train.shape, y_train.shape
X_test.shape, y_test.shape

clf = svm.SVC(kernel='linear', C=1).fit(X_train,
y_train)
clf.score(X_test, y_test)
```



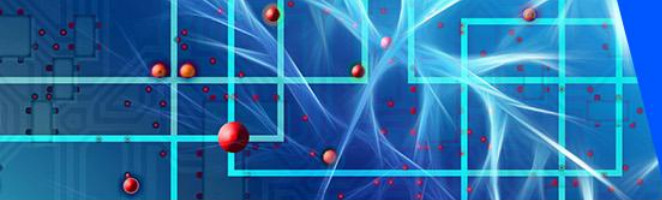
SVM (XIX)

- Cross-Validation (III):
 - Cuando se evalúan diferentes ajustes ("hiperparámetros") para los estimadores, como el ajuste C que debe establecerse manualmente para una SVM, sigue existiendo el riesgo de sobreajuste en el conjunto de pruebas porque los parámetros pueden ajustarse hasta que el estimador tenga un rendimiento óptimo. De este modo, el conocimiento sobre el conjunto de pruebas puede "filtrarse" en el modelo y las métricas de evaluación ya no informan sobre el rendimiento de la generalización.
 - Para resolver este problema, otra parte del conjunto de datos puede mantenerse como el llamado "conjunto de validación": el entrenamiento se lleva a cabo en el conjunto de entrenamiento (training), tras lo cual la evaluación se realiza en el conjunto de validación (validation), y cuando el experimento parece tener éxito, la evaluación final puede realizarse en el conjunto de prueba (test).



SVM (XX)

- Cross-Validation (IV):
 - Sin embargo, al dividir los datos disponibles en tres conjuntos, reducimos drásticamente el número de muestras que pueden utilizarse para el aprendizaje del modelo, y los resultados pueden depender de una determinada elección aleatoria del par de conjuntos (de entrenamiento, de validación).
 - Una solución a este problema es un procedimiento llamado validación cruzada (CV para abreviar). Para la evaluación final debe seguir existiendo un conjunto de pruebas, pero el conjunto de validación ya no es necesario cuando se realiza la CV.
 - En el enfoque básico, denominado CV k-fold, el conjunto de entrenamiento se divide en k conjuntos más pequeños.

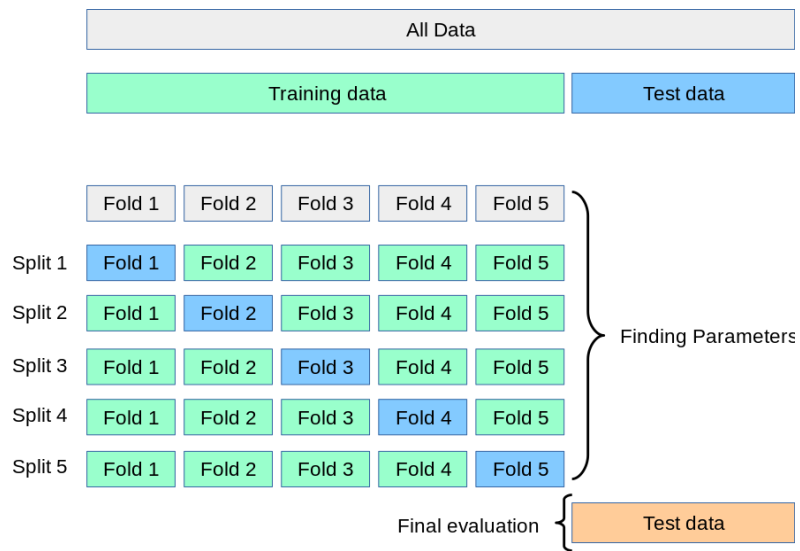


SVM (XXI)

- Cross-Validation (V):
 - K-Fold Validation: Se sigue el siguiente procedimiento para cada uno de los k "pliegues":
 - Se entrena un modelo utilizando uno de los pliegues como datos de entrenamiento
 - El modelo resultante se valida con la parte restante de los datos (es decir, se utiliza como conjunto de pruebas para calcular una medida de rendimiento como la precisión).
 - La medida de rendimiento obtenida mediante la validación cruzada de k-fold es entonces la media de los valores calculados en el bucle.
 - Este enfoque puede ser costoso desde el punto de vista computacional, pero no desperdicia demasiados datos (como ocurre cuando se fija un conjunto de validación arbitrario). Hay problemas donde no existen demasiados datos y desperdiciarlos puede ser un problema.

SVM (XXII)

- Cross-Validation (VI):
 - K-Fold Validation:



https://scikit-learn.org/stable/modules/cross_validation.html

SVM (XXIII)

- Cross-Validation (VII):

- El siguiente ejemplo demuestra cómo estimar la precisión de una SVM de núcleo lineal en el conjunto de datos del iris dividiendo los datos, ajustando un modelo y calculando la precisión 5 veces consecutivas (con diferentes divisiones cada vez):

```
from sklearn.model_selection import cross_val_score
clf = svm.SVC(kernel='linear', C=1)
scores = cross_val_score(clf, X, y, cv=5)
scores

print("%0.2f accuracy with a standard deviation of
%0.2f" % (scores.mean(), scores.std()))
```

- Por defecto, la precisión calculada en cada iteración del CV es la métrica por defecto del estimador. Es posible cambiar esto utilizando el parámetro de scoring.

SVM (XXIV)

- Cross-Validation (VIII):

- Parámetro de scoring:

```
from sklearn import metrics
scores = cross_val_score(
    clf, X, y, cv=5, scoring='f1_macro')
scores
```

- Algunas métricas: Todas las métricas de scoring siguen la convención de que los valores de retorno más altos son mejores que los valores de retorno más bajos.
 - https://scikit-learn.org/stable/modules/model_evaluation.html#scoring-parameter
 - La puntuación F1 puede interpretarse como una media ponderada de la precisión (precision) y la recuperación (recall), donde la puntuación F1 alcanza su mejor valor en 1 y la peor puntuación en 0. La contribución relativa de la precisión y la recuperación a la puntuación F1 son iguales. La fórmula de la puntuación F1 es:
 - $F1 = 2 * (precision * recall) / (precision + recall)$

SVM (XXV)

- Cross-Validation (IX):

- Múltiples parámetro de scoring:

```
from sklearn.model_selection import cross_validate
from sklearn.metrics import recall_score
scoring = ['precision_macro', 'recall_macro']
clf = svm.SVC(kernel='linear', C=1, random_state=0)
scores = cross_validate(clf, X, y, scoring=scoring)
sorted(scores.keys())
```

```
scores['test_recall_macro']
```

- En las métricas, las micro-average y los macro-average calcularán valores ligeramente diferentes y, por tanto, su interpretación difiere. En un problema de multclasificación, un macro calculará la métrica de forma independiente para cada clase y luego sacará la media (por lo tanto, tratará todas las clases por igual), mientras que un micro agregará las contribuciones de todas las clases para calcular la métrica media. En una configuración de clasificación multiclase, es preferible el micropromedio si se sospecha que puede haber un desequilibrio de clases (es decir, puede haber muchos más ejemplos de una clase que de otras).

SVM (XXVI)

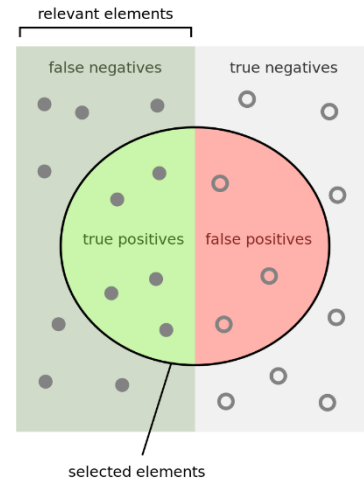
- Cross-Validation (X):

- Seguimos con scoring: Precision y Recall

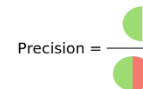
Predicción	Real		
	Gato	Perro	Conejo
Gato	5	2	0
Perro	3	3	2
Conejo	0	1	11

- (TP) True Positive Clase Gato: 5
 - (FP) False Positive Clase Gato: 2
 - (FN) False Negative Clase Gato: 3
 - (TN) True Negative Clase Gato: 17 (resto de casos)
 - Precisión: ¿Qué proporción de identificaciones positivas fue correcta?
 - Recall (Exhaustividad): ¿Qué proporción de positivos reales se identificó correctamente?

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{Recall} = \frac{TP}{TP + FN}$$



How many selected items are relevant?



$$\text{Precision} = \frac{\text{Green}}{\text{Green} + \text{Red}}$$

How many relevant items are selected?



$$\text{Recall} = \frac{\text{Green}}{\text{Green} + \text{White}}$$

SVM (XXVII)

- Cross-Validation (XI):

- Seguimos con scoring:
- La precisión puede ser una métrica engañosa para conjuntos de datos desequilibrados. Considere una muestra con 95 valores negativos y 5 positivos. Clasificar todos los valores como negativos en este caso da una puntuación de precisión de 0.95.

$$\text{Accuracy} = \frac{tp + tn}{tp + tn + fp + fn}$$

- Hay muchas métricas que no sufren este problema. Por ejemplo, la precisión equilibrada normaliza las predicciones positivas y negativas verdaderas por el número de muestras positivas y negativas, respectivamente, y divide su suma por dos:

$$\text{Balanced accuracy} = \frac{TPR + TNR}{2}$$

$$TPR = \frac{TP}{P} = \frac{TP}{TP + FN}$$

$$TNR = \frac{TN}{N} = \frac{TN}{TN + FP}$$

- Para el ejemplo anterior (95 muestras negativas y 5 positivas), la clasificación de todas como negativas da una puntuación de precisión equilibrada de 0,5, que equivale al valor esperado de una conjetura aleatoria en un conjunto de datos equilibrado. La precisión equilibrada puede servir como métrica de rendimiento general para un modelo, independientemente de que las etiquetas verdaderas estén desequilibradas en los datos, suponiendo que el coste de FN sea el mismo que el de FP.
- En scikit, la métrica es: `balanced_accuracy`

SVM (XXVIII)

- Cross-Validation (XII):

- Seguimos con scoring:

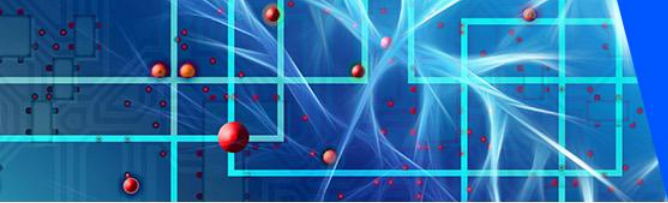
- **balanced_accuracy**

```
from sklearn.model_selection import cross_val_score
clf = svm.SVC(kernel='linear', C=1)
scores = cross_val_score(clf, X, y, cv=5, scoring='balanced_accuracy')
scores
print("%0.2f accuracy with a standard deviation of %0.2f" % (scores.mean(), scores
.std()))
```

- **F₁** score es la media armónica de precisión y recall.

$$F_1 = 2 \cdot \frac{\text{Precisión} \cdot \text{Exhaustividad}}{\text{Precisión} + \text{Exhaustividad}}$$

- ¿Cuándo F1 y cuándo Average? F1 podría ser una mejor medida a utilizar si necesitamos buscar un equilibrio entre Precisión y Recuperación (Recall), y hay una distribución de clases desigual (gran número de Negativos Reales).
 - F-scoring se ha utilizado ampliamente en procesamiento del lenguaje natural así como en reconocimiento de entidades con nombre y la segmentación de palabras.



Ejercicio adicional (I)

Ejercicio 1: Clasificación de Flores Iris

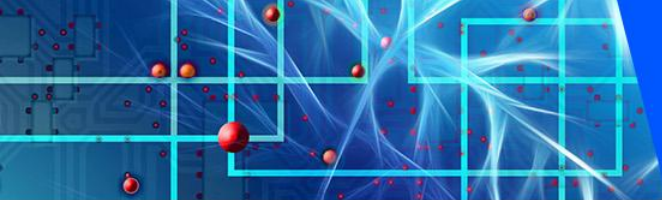
El conjunto de datos Iris es un clásico en el aprendizaje automático y contiene 150 observaciones de flores de 3 especies diferentes: Setosa, Versicolor y Virginica. Cada observación tiene 4 características numéricas: longitud y ancho del sépalo, longitud y ancho del pétalo.

Objetivo:

Utiliza un modelo de SVM para clasificar las flores en una de las 3 especies utilizando sus características.

Pasos:

- **Cargar el conjunto de datos Iris:** El conjunto de datos Iris está disponible en muchas bibliotecas de Python, como `sklearn.datasets`.
- **Dividir los datos en conjunto de entrenamiento y prueba:** Utiliza el 80% de los datos para entrenar el modelo y el 20% para probarlo.
- **Normalizar los datos:** Los SVM son sensibles a la escala de las características, por lo que es recomendable normalizar las variables numéricas (por ejemplo, utilizando `StandardScaler`).
- **Entrenar el modelo SVM:** Entrena un modelo de SVM para clasificar las flores en sus respectivas especies. Puedes usar el kernel lineal o un kernel no lineal (como RBF) para este caso.
- **Evaluar el modelo:** Mide la precisión del modelo en el conjunto de prueba. Genera la matriz de confusión para ver cómo de bien el modelo ha clasificado cada una de las especies.



Ejercicio adicional (I)

Ejercicio 2: Clasificación de Dígitos Escritos a Mano (MNIST)

El conjunto de datos MNIST contiene imágenes de dígitos escritos a mano (del 0 al 9) y es ampliamente utilizado en problemas de clasificación. El objetivo será utilizar SVM para clasificar estos dígitos basándose en las características de las imágenes.

Objetivo:

Entrena un modelo SVM para clasificar imágenes de dígitos escritos a mano en una de las 10 categorías (dígitos del 0 al 9).

Pasos:

- **Cargar el conjunto de datos MNIST:** El conjunto de datos MNIST está disponible en la librería `sklearn.datasets`. Este conjunto de datos incluye imágenes de 28x28 píxeles que representan los dígitos del 0 al 9.
- **Dividir los datos en conjunto de entrenamiento y prueba:** Divide el conjunto de datos para entrenar el modelo con el 80% de los datos y probar con el 20% restante.
- **Normalizar los datos:** Normaliza los valores de los píxeles de las imágenes para que estén en el rango adecuado (por ejemplo, entre 0 y 1). Esto ayuda a que el SVM funcione mejor.
- **Entrenar el modelo SVM:** Entrena un modelo SVM (puedes empezar con un kernel lineal y luego probar otros como RBF).
- **Evaluar el modelo:** Evalúa el rendimiento del modelo utilizando el conjunto de prueba. Calcula la precisión y genera una matriz de confusión para ver en qué dígitos el modelo comete más errores.

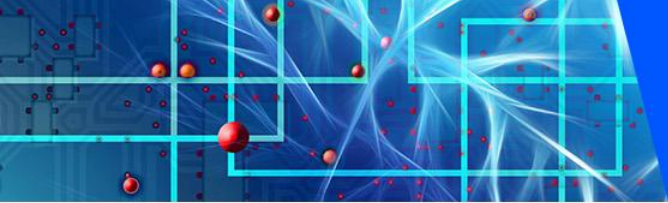
Ejercicio adicional (I)

Ejercicio 3: Estimación de ingresos

- Utilizando el dataset de ingresos por adulto (<https://archive.ics.uci.edu/ml/datasets/Adult>), que tiene los siguientes atributos y clases:
- >50K, <=50K.
 - age: continuous.
 - workclass: Private, Self-emp-not-inc, Self-emp-inc, Federal-gov, Local-gov, State-gov, Without-pay, Never-worked.
 - fnlwgt: continuous.
 - education: Bachelors, Some-college, 11th, HS-grad, Prof-school, Assoc-acdm, Assoc-voc, 9th, 7th-8th, 12th, Masters, 1st-4th, 10th, Doctorate, 5th-6th, Preschool.
 - education-num: continuous.
 - marital-status: Married-civ-spouse, Divorced, Never-married, Separated, Widowed, Married-spouse-absent, Married-AF-spouse.
 - occupation: Tech-support, Craft-repair, Other-service, Sales, Exec-managerial, Prof-specialty, Handlers-cleaners, Machine-op-inspct, Adm-clerical, Farming-fishing, Transport-moving, Priv-house-serv, Protective-serv, Armed-Forces.
 - relationship: Wife, Own-child, Husband, Not-in-family, Other-relative, Unmarried.
 - race: White, Asian-Pac-Islander, Amer-Indian-Eskimo, Other, Black.
 - sex: Female, Male.
 - capital-gain: continuous.
 - capital-loss: continuous.
 - hours-per-week: continuous.
 - native-country: United-States, Cambodia, England, Puerto-Rico, Canada, Germany, Outlying-US(Guam-USVI-etc), India, Japan, Greece, South, China, Cuba, Iran, Honduras, Philippines, Italy, Poland, Jamaica, Vietnam, Mexico, Portugal, Ireland, France, Dominican-Republic, Laos, Ecuador, Taiwan, Haiti, Columbia, Hungary, Guatemala, Nicaragua, Scotland, Thailand, Yugoslavia, El-Salvador, Trinidad&Tobago, Peru, Hong, Holand-Netherlands.



<https://pixabay.com/photos/money-dollars-currency-to-save-3523131/>



Ejercicio adicional (II)

Se pide:

1. Cargar el juego de entrenamiento y el juego de test:
`https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data`
`https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.test`
2. Filtrar los atributos: age, education-num, race, sex y hours-per-week (columnas 0, 4, 8, 9, 12) y el objetivo: >50K y <=50K
3. Como algunas variables son alfanuméricas, se pide categorizarlas. Como deben ser igual en train y en test, podéis utilizar un diccionario.
Por ejemplo: `dictSex = { "Female": 0, "Male": 1 }`
`dictSex["Female"]` devolverá 0.
También hay que categorizar la clase del objetivo (`dictRenta = {"<=50K":0, "<=50K.":0, ">50K": 1, ">50K.": 1}`)
4. Hacer un shuffle de los datos y convertirlos a tipo float32 para poder trabajar con el modelo.
5. Utilizando ExtraTrees, identificar las 3 características que más contribuyen al modelo y filtrarlas.
6. Entrenar el modelo SVM con GRID sobre los tres atributos más destacados. Como serán muchos datos, podéis quedaros con las primeras 200 filas.
Por ejemplo: `param_grid = {'C': [1, 10], 'gamma': [0.01, 0.001], 'kernel': ['linear', 'rbf']}`
7. Evaluar el modelo con los parámetros obtenidos con el grid (con los datos de test)
8. Mostrar una matriz de confusión de los resultados.